

ARM PrimeCell

Color LCD Controller (PL110)

Integration Manual

ARM PrimeCell Color LCD Controller (PL110) Integration Manual

© Copyright ARM Limited 1999 – 2003. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is ARM Confidential (Distributed to ARM staff, product licensees & NDA signatories only).

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell Color LCD Controller

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	1
1.1	Change history	1
1.2	References	1
1.3	Terms and abbreviations	1
2	SCOPE	2
3	INTRODUCTION	2
4	INTEGRATION	3
4.1	Design Flow Supported	3
4.2	Signal Connectivity	4
4.3	Bus Interconnection	6
4.4	Endianness	7
4.5	Customisation	7
4.5.1	CLCD Controller Parameters	7
4.6	Synthesis	7
4.6.1	Setup Scripts	7
4.6.2	Command Script	8
4.6.3	Constraint Scripts	8
4.7	Integration with AMBA components	9
4.8	Interrupt Sources	9
4.9	Clocking	10
4.10	Clock Tree synthesis	10
4.11	Resets	11
4.12	Reset Buffering	12
4.13	Synchronizers	12
4.14	Test Methodologies	13
4.14.1	Scan Insertion and ATPG	13

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change
A01	29 July, 1999	First draft.
A	6 August, 1999	Initial Release
B	15 September 1999	“Resets” section upgraded, “Reference” section corrected plus a few minor typing modifications.
C	28 March 2003	r1p2 release. Updated Section 4.2 Signal Connectivity; Figure 4.8-1 Interrupt Structure; minor typos

1.2 References

This document refers to the following documents.

Ref	Doc No	Title
1	ARM IHI 0011	AMBA Specification (Rev 2.0)
2	ARM DDI 0161	ARM PrimeCell CLCD Controller (PL110) Technical Reference Manual
3	PL110 DDES 0000C	ARM PrimeCell Color LCD Controller (PL110) Design Manual

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AMBA	Advanced Micro-controller Bus Architecture
AHB	Advanced High-performance Bus
DMA	Direct memory Access
FIFO	First-in-First-out
ASIC	Application Specific Integrated Circuit
ATPG	Automatic Test Pattern Generation
IP	Intellectual Property
RTL	Register Transfer Level
STA	Static Timing Analysis
CLCD	Color Liquid crystal display

2 SCOPE

This document describes how the ARM PrimeCell Color LCD Controller may be integrated into a typical industry standard ASIC design flow.

3 INTRODUCTION

The Color LCD Controller is a part of a library of reusable IP blocks, which can be more easily integrated into a typical ASIC design flow.

A carefully chosen set of design rules has been followed to allow faster integration in most ASIC design flows using standard synthesis and test tools. The implementation can easily be customized to suit specific customer requirements.

For further information on the AMBA AHB, refer to the AMBA Specification [Ref.1].

For detailed technical information and programming data on the ARM PrimeCell Color LCD Controller block, refer to the ARM PrimeCell Color LCD Controller (PL110) Technical Reference Manual [Ref.2].

The ARM PrimeCell Color LCD Controller (PL110) Design Manual [Ref. 3] contains information about the implementation and design of the Color LCD Controller block that may be needed for optional customization of this block.

4 INTEGRATION

4.1 Design Flow Supported

The ARM PrimeCell Color LCD Controller has been designed to allow integration with other IP blocks using typical ASIC design flows. Although consideration has been given to minimisation of area (gate count) and power-consumption, ease of integration has been given higher priority to aid design reuse.

The following sections of this document address issues that may arise during ASIC system-level integration of the ARM PrimeCell Color LCD Controller block.

An example ASIC design flow is described below:

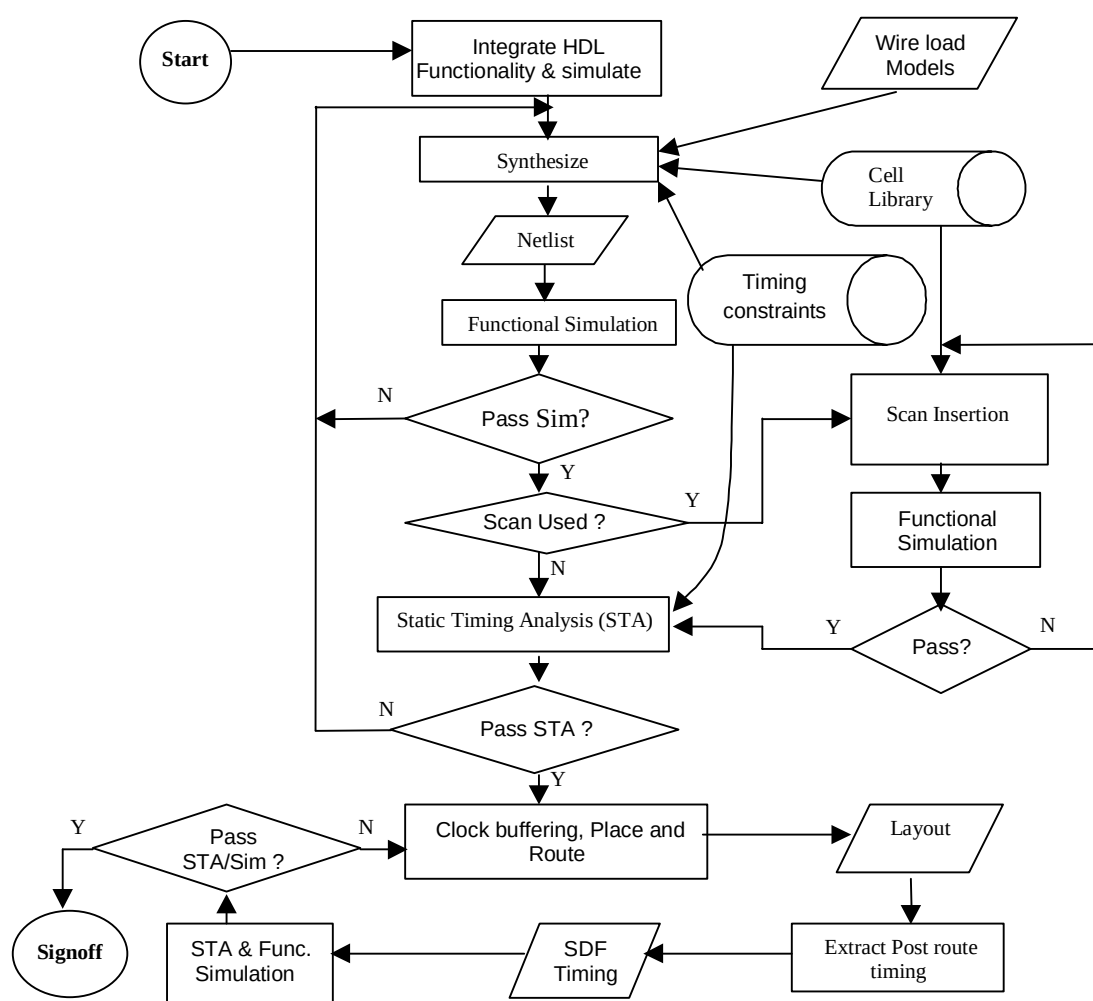


Figure 4.1-1: Example ASIC Design Flow supported

As an alternative to dynamic functional simulation it is also possible to use logic equivalence checking.

4.2 Signal Connectivity

The following tables describe the I/O pins for the ARM PrimeCell Color LCD Controller block. For further information refer to ARM PrimeCell Color LCD Controller (PL110) Technical Reference Manual [Ref.2].

Signal name	Type	Source/destination	Description
HCLK	IN	AHB bus	Bus Clock
HRESETn	IN	AHB bus	Bus reset signal (active low).
HSELCLCD	IN	Decoder	Device select signal
HADDRS[11:2]	IN	AHB bus	Address bus
HTRANS[1:0]	IN	AHB bus	Indicates the type of the current transfer.
HWRITES	IN	AHB bus	When HIGH this signal indicates a write transfer and when LOW a read transfer.
HWDATAS[31:0]	IN	AHB bus	Write data bus
HRDATAS[31:0]	OUT	AHB bus	Read data bus
HREADYINS	IN	AHB bus	When HIGH this signal indicates that a transfer has finished on the bus.
HREADYOUTS	OUT	AHB bus	When HIGH this signal indicates that the slave is ready for the next transfer. This signal may be driven LOW to extend the transfer.
HRESPS[1:0]	OUT	AHB bus	This signal provides additional information on the status of a transfer(OKAY, ERROR)

Table 4.2-1 AHB signals (Slave Interface)

Signal name	Type	Source/destination	Description
HADDRM[31:0]	OUT	AHB bus	Address bus
HTRANSM[1:0]	OUT	AHB bus	Indicates the type of the current transfer
HWRITEM	OUT	AHB bus	Read/ Write signal. The Master interface performs only Read access.
HSIZEM[2:0]	OUT	AHB bus	Indicates the size of the transfer. Which is typically word access(32-bit)
HBURSTM[2:0]	OUT	AHB bus	Indicates if the transfer forms a part of the burst. Supports only incremental burst.
HRDATAM[31:0]	IN	AHB bus	Read data bus
HREADYINM	IN	AHB bus	When HIGH this signal indicates that a transfer has finished on the bus.
HRESPM[1:0]	IN	AHB bus	Response from the slave.
HPROTM[3:0]	OUT	AHB bus	The protection signal provides an additional information about a bus access. This signal is always driven with a value of "0001" (user data access).
HLOCKM	OUT	Arbiter	When HIGH this signal indicates that the master requires locked access. This signal is always driven for non-locking mode.
HBUSREQM	OUT	Arbiter	Bus Request
HGRANTM	IN	Arbiter	Bus Grant

Table 4.2-2 AHB Signal (Master Interface)

Signal name	Type	Source/destination	Description
CLCDCLK	IN	Clock MUX	Clock for LCD Controller main blocks.
nCLCDCLK	IN	Clock MUX	Inverted CLCDCLK for LCD Controller main blocks.
CLCDCLKSEL	OUT	Clock MUX	Clock source select. Bit 5 of LCDTiming register 2 drives this signal.
nCLCLKRESET	IN	Reset MUX	System Reset signal synchronised to the CLCDCLK domain
SCANENABLE	IN	Test Controller	CLCD scan enable signal for all clock domains
SCANINHCLK	IN	Test Controller	CLCD input scan signal for the HCLK domain

SCANINCLCDCLK	IN	Test Controller	CLCD input scan signal for the CLCDCLK domain
SCANINnCLCDCLK	IN	Test Controller	CLCD input scan signal for the nCLCDCLK domain
SCANOUTHCLK	OUT	Test Controller	CLCD output scan signal for the HCLK domain
SCANOUTCLCDCLK	OUT	Test Controller	CLCD output scan signal for the CLCDCLK domain
SCANOUTnCLCDCLK	OUT	Test Controller	CLCD output scan signal for the nCLCDCLK domain
CLCDMBEINTR	OUT	Interrupt Controller	Bus error interrupt
CLCDFUFINTR	OUT	Interrupt Controller	DMA FIFO underflow interrupt
CLCDLNBUINTR	OUT	Interrupt Controller	Next base address update interrupt
CLCDVCOMPINTR	OUT	Interrupt Controller	Vertical compare interrupt
CLCDINTR	OUT	Interrupt Controller	Combined interrupt

Table 4.2-4 On-chip signals

Signal name	Type	Source/destination	Description
CLPOWER	OUT	PAD	LCD Panel Power enable
CLLP	OUT	PAD	Line sync Pulse(STN) / Horizontal Sync pulse(TFT)
CLCP	OUT	PAD	LCD panel clock
CLFP	OUT	PAD	Frame pulse(STN) / Vertical Sync pulse(TFT)
CLAC	OUT	PAD	STN AC bias drive or TFT data enable output
CLD[23:0]	OUT	PAD	LCD panel data
CLLE	OUT	PAD	Line End signal

Table 4.2-3 signals to I/O pad cells

4.3 Bus Interconnection

The Color LCD Controller block interfaces with the AMBA AHB interface using separate uni-directional read (output) and write (input) data buses HRDATA and HWDATA. For further information refer to the AMBA Specification [Ref.1].

The data bus may be interfaced to the Bus Masters either using multiplexers or logical OR'ing of data buses from other AHB peripheral blocks.

4.4 Endianness

The CLCD Controller is designed for word access only. Hence it is the responsibility of the system to ensure that only word accesses are made to the CLCD controller. And if the master attempts byte or half-word read access, the CLCD controller always returns the full word.

4.5 Customisation

The ARM PrimeCell Color LCD Controller can be modified to suit specific user configurations, but the full benefit of a fast integration will be only obtained when the design is used without modification. In the event of modification, it is the user's responsibility to ensure that the design is fully functional by appropriate amendment of the functional test program and the testbenches where required. Information to support such modification is included in the Color LCD Controller (PL110) Design Manual [Ref.3].

4.5.1 CLCD Controller Parameters

The user is required to set the following parameters in the ClcdConfig module to suitable values before synthesizing the design.

FIFO_TYPE: This specifies the type of the DMA FIFO buffer. This parameter should be set to 1 for d-type FIFO.

Previous versions of the PL110, supported setting this parameter to 0 to invoke a RAM cell (TPRAM) instead of the d-type. Instantiation of a TPRAM is not supported by ARM for this release. The default value is 1.

FIFO_DEPTH: This parameter specifies the DMA FIFO depth. The minimum allowable value is 8 and can take values of 16, 32, 64 etc up to a maximum value of 512 words. The default value is 16.

PTR_SIZE: This parameter specifies the DMA FIFO read/write pointer size. This is computed as the logarithm of FIFO_DEPTH to base 2. The default value is 4.

PAL_RAM_READ_STRTCH: This parameter specifies the LCD palette RAM read cycle delay. If this is set to 1'b1 the read cycle is stretched by 1-HCLK for Palette RAM and the DMA FIFO RAM access. The default value is 1'b0.

4.6 Synthesis

The Color LCD Controller block has been synthesized using the Synopsys Design Compiler synthesis tool with the scripts provided. The final area and gate count that will be achieved is dependent on the target cell library used.

A README text file in the synopsys/ directory provides details of the synthesis directory structure used.

The synthesis scripts consist of the following three main categories:

4.6.1 Setup Scripts

Scripts that are common to several designs are located in the global/synopsys_shared/scr_common directory. The following files need several parameters or variables to be changed in order to target a different cell library.

The setup script, 'setup.scr' defines the following:

- Synopsys variable settings
- UNIX directory path for Synopsys read and write caches

The 'global.scr' script may need modifying to define the following:

- target area goals

- maximum transition costs

The 'library_avanti_cb18_v1.0.scr' script is included by the master compile script to specify the target library and the cells in the target library that should not be used. It is recommended that an equivalent file be created for the target library as per library vendor or customer requirements. It is also a practice to sometimes prevent the synthesis tool from inferring lower drive cells for timing reasons. The user must ensure that the 'library_avanti_cb18_v1.0.scr' file is modified and renamed appropriately for the chosen technology library and included by the master compile script.

The '\$LIB_SYNOP' directory must contain the files, or links to the target cell library files, in Synopsys format.

4.6.2 Command Script

The master compile command scripts located in the synopsys/scripts directory, analyses the corresponding HDL source before mapping and optimizing to a specific target technology cell library. These scripts include setup and constraint scripts to set timing and design rule constraints on the design, prior to optimization. The compiled design is written out both in native Synopsys .db format and in VHDL/Verilog netlist format. Pre-route (post-synthesis) timing information for back-annotation is written out to SDF files and various reports for timing and violations are also written out. Additionally the run-time log is redirected into a separate log/ directory.

The scan-insertion script Clcd_scan.cmd in the synopsys/scripts directory performs scan insertion for the design.

Prior to synthesis, several variables need to be set. The scripts use environment variables, which are assigned to synthesis variables in the scripts. The environment variables provide flexibility without requiring editing of the scripts. The following environment variables are used:

- \$PERIPH is assigned to 'topname' to define the name of the top level module in the design.
- \$HDL_SOURCE is assigned to 'hdl' to define the input HDL source. HDL_SOURCE should be set to either vhdl or verilog in order to read in the VHDL or Verilog source RTL files.
- \$HDL_NETL is assigned to 'hdl_out' to define the output format for netlist and SDF files. It can be set to either vhdl or verilog to generate either VHDL or Verilog netlists.

If the environment variables are not required, then the scripts can be modified to directly assign values.

4.6.3 Constraint Scripts

- Operating conditions and signal load/driver values should be set for the chosen technology library in a file which is equivalent to the 'library_avanti_cb18_v1.0.scr' file.
- Timing constraints for AHB bus signals are set via the amba_params.scr, ahb_master.scr and the ahb_slave.scr files. As delivered, the timings are set for a 166 MHz clock with 50% duty cycle. These timings should be adjusted according to the target frequency required. The above-mentioned files have been written using parameters and only requires amendment of the clock frequency with all other bus parameters (except clock skew) scaling accordingly.

Clock skew needs to be set explicitly as minus (worst case) and plus (best case) uncertainty. The following default values have been used as preliminary values prior to layout:

- the minus uncertainty value is set at 2.5% of the clock period.
- the plus uncertainty value is set at 40% of the minus uncertainty value.

The plus uncertainty derives from best case condition and it is less than the minus uncertainty value which derives from worst case condition.

The timing parameters assume the following bus timing budgets:

- 30 % input setup on all input ports including those on the AHB Slave ports and the AHB Master ports.
 - 40 % output valid delay on all outputs including those on the AHB Slave ports and the AHB Master ports.
- Timing constraints for non-AMBA CLCD-specific signals are set via the Clcd.scr file. Sample input and output delay constraints are specified for these signals. The minimum delays are set so that the synthesis tool does not insert buffers to fix hold violations that do not exist in reality.

- d) Timing exceptions files are used to specify false paths. The exception file corresponding to each of the block with some explanations are enumerated as follows :
 - Top level CLCD : the exceptions are defined in the Clcd_exceptions.scr file

Note The synthesis scripts assume all input clocks have a 50% duty cycle.

4.7 Integration with AMBA components

The Color LCD Controller block is designed to be integrated into an AMBA system via the AHB (Advanced High Performance Bus.). The CLCD controller has two independent AHB bus connections, one each for DMA access and register programming. The AHB interface for DMA access is master only interface while the AHB interface for register and Palette programming is slave only interface. Both these buses can be combined to make a single AHB bus with both master and slave capabilities for DMA and controller programming respectively. It is the user's responsibility to design the DMA interface since this design would depend on the interface of the specific DMA peripheral.

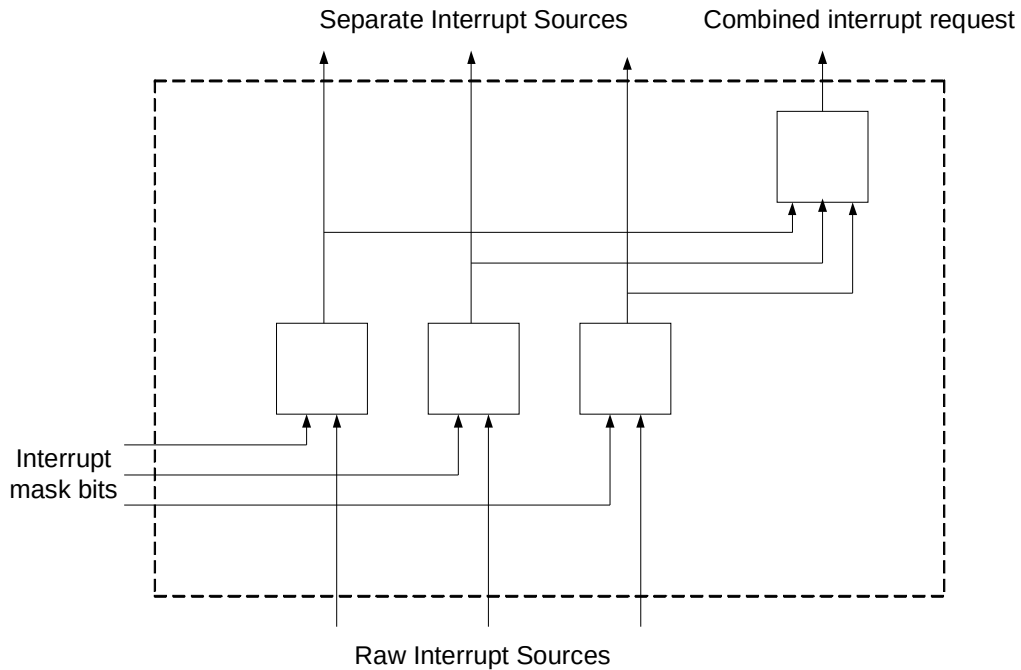
4.8 Interrupt Sources

The CLCD Controller block provides flexibility in choice of an interrupt handling strategy by providing both individual and combined interrupts. Four interrupting conditions (DMA FIFO underflow, Lcd next base address update, Vertical compare status and AHB master error) are indicated by separate interrupt lines. These conditions are also logically OR'ed together to provide a single combined interrupt.

The interrupt architecture could use one of the two schemes below:

- 1 Each peripheral provides a single combined interrupt source to the system interrupt controller, which is the OR function of the internal masked sources. The system interrupt controller may then provide another level of masking on a per peripheral basis.
- 2 Each peripheral passes all of its individual sources to the system interrupt controller which would also provide the mask registers for each individual source. In this way a global interrupt service routine would be able to read the entire set of sources from one wide register in the system interrupt controller.

Figure 4.8-1 illustrates how a typical peripheral block outputs both the individual masked interrupts as well as a combined output.

**Figure 4.8-1: Interrupt structure**

The former system has the advantage of allowing modular device drivers which always know where to find the interrupt source control register bits. The latter system has an advantage where a more integrated interrupt service routine is required for the system and it would be especially attractive where the time to read from the peripheral registers is significant compared to the CPU clock speed in a real time system.

The CLCD Controller block has been designed to support system architectures which use either type of interrupt controller since the overhead for doing so is small.

Software may be implemented with either separate interrupt service routines for each interrupt or with a single interrupt service routine which polls each of the two interrupt status bits to determine the interrupt source. Each of these interrupt sources can be masked individually within the CLCD Controller itself, thus complementing the mask bits provided in the interrupt controller. The ARM core has two interrupts – nIRQ and nFIQ. Normally the CLCD Controller would route the interrupt source to the nIRQ input of the ARM core.

4.9 Clocking

The CLCD controller has three clock inputs. The AHB bus clock HCLK is used for AHB slave, AHB master and the DMA FIFO. The main clock input CLCDCLK is used to clock the LCD control logic. This clock can be an asynchronous clock or HCLK. The output CLCDCLKSEL from the controller is nothing but a programmable register bit that can be used to control the multiplexer select the main clock (CLCDCLK) on which the LCD main logic operates. The inverted main clock (nCLCDCLK) is used in the LCD panel clock generation module to clock one flip-flop which otherwise needs falling edge clocking if CLCDCLK is used.

4.10 Clock Tree synthesis

Many different design methodologies are available for clock tree insertion and synthesis. The default CLCD Controller synthesis scripts cause the Synopsys tools to output a netlist with no buffering on the clock nets. It is assumed that clock buffering will be performed by a separate tool, such as place & route tools, which understand placement, further on in the backend process. It is possible to modify the scripts if this is not the preferred

approach or if this is not compatible with the design flow being used. It is recommended however that clock tree insertion should always be performed during the place and route stage and not during synthesis.

4.11 Resets

The CLCD Controller block has one reset pin for each of its internal clock domains. The system bus reset signal HRESETn, resets the AHB interface. A separate reset pin nCLCLKRESET, is used to reset the CLCDCLK domain logic in the CLCD Controller. This signal needs to be provided by a separate Reset Controller, and it must be de-asserted synchronously to CLCDCLK. The resets can be asserted asynchronously even if no clocks are present to ensure that signals from bi-directional I/O cells have defined values during reset, thus avoiding static power consumption problems. While re-synchronization of HRESETn to CLCDCLK could have been done within the CLCD Controller block itself, the function was moved out to eliminate internally generated resets.

Figure 4-11-1 below illustrates a method of generating nCLCLKRESET from HRESETn, such that it will be asserted asynchronously and de-asserted synchronous to CLCDCLK. The reset signal nCLCLKRESET is de-asserted two CLCDCLK cycles after HRESETn is de-asserted to prevent metastability problems

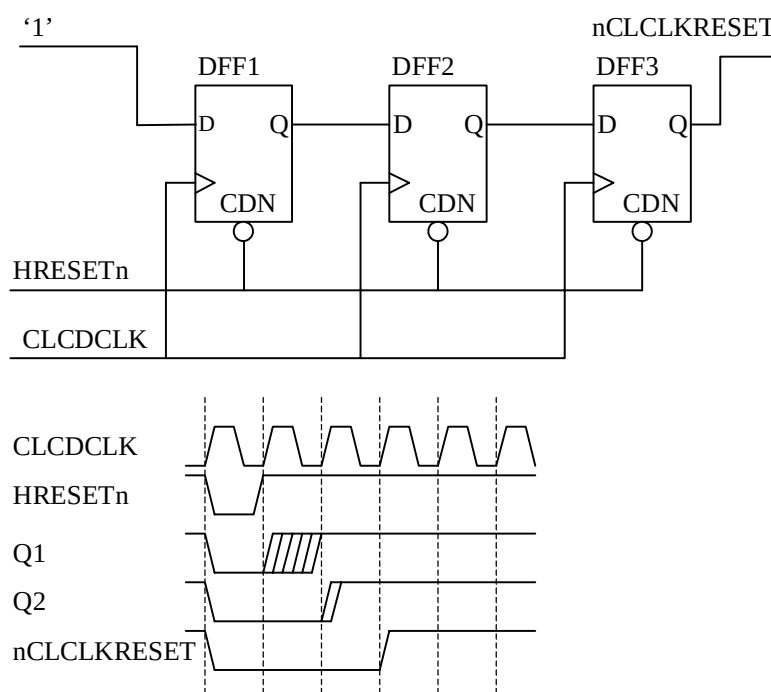


Figure 4.11-1 A method of generating nCLCLKRESET

The method of generating nCLCLKRESET in Figure 4.11-1 could experience simulation problems depending on the cell library modeling. The Q output of DFF3 should not be metastable when setup/hold violation occurs due to de-assertion of HRESETn, since both D and Q are both '0', however the DFF model may assign 'X'.

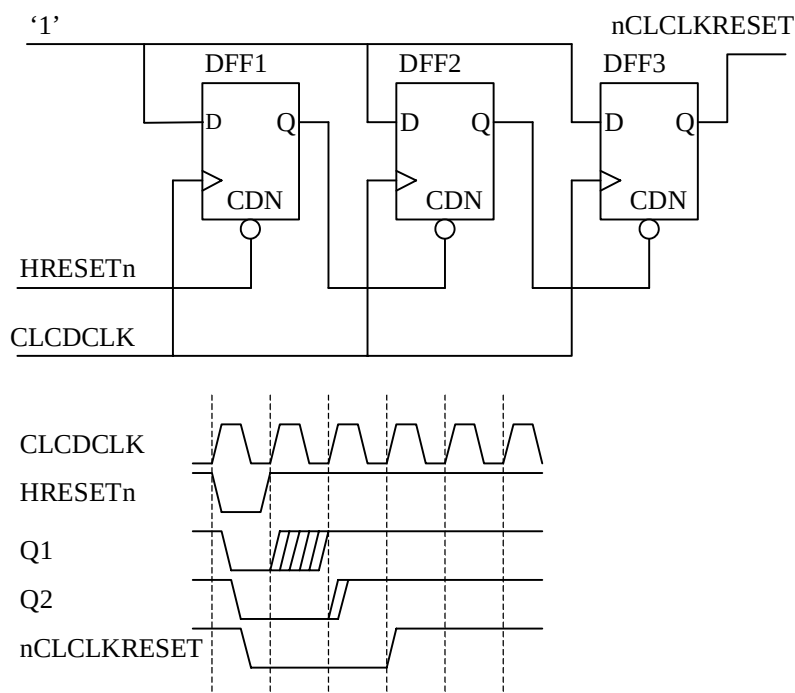


Figure 4.11-2 An alternative method of generating nCLCLKRESET

An alternative method of generating nCLCLKRESET is therefore illustrated in Figure 4.11-2 which achieves the same functionality. This method does use internally generated resets, which can present problems in Static Timing Analysis, scan insertion and ATPG, but they are limited to the reset controller at the top-level of the design hierarchy.

4.12 Reset Buffering

One of several available methods should be used to ensure correct buffering of the reset nets. The reset signals could be assigned infinite drive strength allowing a chip-level integration process to ensure balanced reset trees are used at the top-level of the chip or propagating the tree down to all end points. Another alternative is to apply timing constraints and ensure that block synthesis instantiates cells with adequate drive strength in each block.

4.13 Synchronizers

Single bit signals between the two clock domains are synchronized by two series flip-flops inferred during synthesis from standard HDL templates. Multiple bit vector signals use an update mechanism with synchronized single bit control signals to avoid meta-stability problems.

Some technology libraries offer meta-stability hardened D flip-flops specifically for implementing synchronizers. All the synchronizers in the CLCD Controller block have been isolated to the sub-blocks ClcdSyncHCLK and ClcdSyncCLCDCLK. This allows the synthesis scripts to be modified so that these blocks could be synthesized separately to use the synchronizer flip-flop cells using the Synopsys Design Compiler `set_prefer` command. Further synthesis of these sub-blocks can then be prevented using the `set_dont_touch` command. The rest of the CLCD Controller design is then synthesized after placing a `set_dont_use` command on the synchronizer cells. This method also offers the opportunity to simulate the gate-level netlist with asynchronous clocks HCLK and CLCDCLK and eliminate the expected setup and hold violations on the synchronizers from the simulation transcript, since the synchronizers cells can be modified to remove 'X' propagation.

4.14 Test Methodologies

The CLCD Controller is designed to be tested using Scan insertion and ATPG.

4.14.1 Scan Insertion and ATPG

It is assumed that users will already be familiar with the design flow for scan insertion and ATPG, so only a brief discussion of integration issues arising are given below.

The CLCD Controller design fully supports this scan test and ATPG methodology. Only the rising edges of clocks are used in the design.

If scan insertion is to be performed post-synthesis, it should be done once the netlist has been proven to be logically equivalent either by dynamic simulation using the functional test vectors or use of equivalence checking. Scan insertion is then performed and the logical equivalence again proven. For scan insertion using a two-pass approach, the RTL is compiled into a netlist without scan cells in the first pass and in the second pass, the normal storage elements are replaced with scan cells and the scan chains are formed. Alternatively, a one-pass approach may be adopted for synthesis where the RTL is compiled to create netlist with a scan test D flip-flops and provisionally connected scan chains. After scan insertion logical equivalence should again be proven since gate-level timing will now be different.

The main advantage of the scan Insertion and ATPG approach is in cases where significant modifications have been made to the CLCD Controller block design by the user, or when most other user-generated parts of the chip use scan testing. Another advantage is that very high fault coverage can be achieved (99%).

Extraction of test vectors is done by the system designer once the whole chip design has been integrated. The quoted fault coverage (excepting variation resulting from the use of a sparse cell library) will be met by extracting test vectors around the complete chip while running automatically generated vectors (ATPG).